BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah mengubah cara masyarakat menyimpan dan mempertukarkan nilai. Setelah melewati era barter, uang komoditas, dan uang fiat yang dijamin oleh otoritas pemerintah, kini muncul kelas aset baru berupa aset digital dan mata uang kripto (*cryptocurrency*) yang beroperasi tanpa otoritas pusat. Kehadiran aset ini menandai pergeseran paradigma dalam sistem keuangan global, di mana kecepatan, transparansi, dan desentralisasi menjadi nilai utama dalam aktivitas ekonomi yang semakin terhubung (Muhammad, 2024).

Bitcoin merupakan mata uang kripto pertama yang diperkenalkan pada tahun 2008 melalui makalah berjudul "*Bitcoin: A Peer-to-Peer Electronic Cash System*" oleh individu atau kelompok anonim dengan nama samaran Satoshi Nakamoto (Nakamoto, 2008), dan mulai beroperasi pada tahun 2009. Seluruh transaksi Bitcoin dicatat dan diverifikasi secara kolektif oleh partisipan jaringan dalam buku besar terdistribusi yang disebut *blockchain*, sehingga tidak memerlukan perantara tepercaya seperti bank (Muhammad, 2024). Dalam perkembangannya, Bitcoin bertransformasi dari sebuah eksperimen teknologi menjadi kelas aset investasi yang diakui secara global. Kenaikan harganya yang mencapai lebih dari 1.700% dalam rentang 2017 hingga 2022 telah menarik minat investor dari kalangan ritel hingga institusional (Rizki *et al*., 2023). Pada periode yang diamati dalam penelitian ini saja, harga penutupan Bitcoin bergerak dari titik terendah 25.162,70 USD pada 12 September 2023 hingga titik tertinggi 124.776,68 USD pada 7 Oktober 2025.

Di balik potensi keuntungannya, Bitcoin memiliki karakteristik utama berupa volatilitas harga yang sangat tinggi. An *et al*. (2023) mencatat bahwa harga Bitcoin pernah turun sekitar 50% dari puncaknya pada tahun 2021, sedangkan Hansun *et al*. (2022) menegaskan bahwa kenaikan dan penurunan mendadak yang dipicu oleh banyak faktor membuat nilai aset kripto di masa depan sulit diprediksi. Dari sudut pandang ilmu aktuaria yang berfokus pada pengukuran dan pengelolaan risiko, volatilitas yang tinggi berarti ketidakpastian nilai aset yang besar dan berdampak langsung pada pengukuran risiko pasar, penentuan cadangan, serta keputusan investasi. Wijaya *et al*. (2025) menunjukkan bahwa hasil peramalan harga Bitcoin dapat dipadukan dengan ukuran risiko seperti *Value at Risk* (VaR) dan *Expected Shortfall* (ES) untuk menakar potensi kerugian investasi. Oleh karena itu, model prediksi harga yang akurat merupakan masukan kuantitatif yang penting dalam pengelolaan risiko aset kripto.

Secara unik, teknologi *blockchain* yang melahirkan Bitcoin juga menyediakan sumber data yang berpotensi menjelaskan pergerakan harganya. Karena seluruh transaksi tercatat secara permanen dan terbuka, berbagai metrik

jaringan seperti pendapatan penambang (*miners revenue*), tingkat kesulitan penambangan (*difficulty*), laju *hash* (*hash rate*), jumlah transaksi, dan jumlah alamat unik dapat diamati setiap hari. Metrik-metrik ini dikenal sebagai informasi *blockchain* atau data *on-chain* yang mencerminkan kesehatan jaringan, aktivitas ekonomi, dan perilaku partisipan pasar (Chrisna *et al*., 2024). Penelitian terkini menunjukkan bahwa data *on-chain* bermanfaat sebagai fitur prediktif. Kim *et al*. (2022) membangun model LSTM berbasis mekanisme *attention* untuk kelompok variabel *on-chain*, An *et al*. (2023) memperoleh MAPE sebesar 3,993% dan koefisien determinasi sebesar 0,976 menggunakan LSTM dengan sepuluh variabel *on-chain*, sedangkan Omole & Enke (2024) memanfaatkan 87 metrik *on-chain* untuk memprediksi arah pergerakan harga Bitcoin.

Data harga Bitcoin dan informasi *blockchain* bersifat nonlinier dan nonstasioner, sehingga metode runtun waktu klasik seperti *Autoregressive Integrated Moving Average* (ARIMA) yang mengasumsikan hubungan linier sering kali kurang memadai (Seabe *et al*., 2023). Metode *deep learning*, khususnya keluarga *Recurrent Neural Network* (RNN), muncul sebagai alternatif karena mampu mempelajari pola nonlinier dan ketergantungan temporal. Salah satu varian RNN yang paling banyak digunakan adalah *Long Short-Term Memory* (LSTM) yang dirancang untuk mengatasi masalah *vanishing gradient* (Hochreiter & Schmidhuber, 1997). Putra (2024) menunjukkan bahwa LSTM menghasilkan akurasi prediksi sebesar 98,8%, jauh di atas ARIMA yang hanya 66,4%, sedangkan penelitian Rizkilloh & Widiyanesti (2022) serta Nirraca & Hartati (2024) juga membuktikan kemampuan LSTM dalam memprediksi harga aset kripto.

Selain LSTM, terdapat *Gated Recurrent Unit* (GRU) yang diperkenalkan oleh Cho *et al*. (2014) sebagai penyederhanaan LSTM. GRU hanya memiliki dua gerbang dan tidak memiliki *cell state* terpisah, sehingga jumlah parameternya lebih sedikit. Namun, penelitian dalam lima tahun terakhir belum memberikan kesimpulan yang seragam mengenai model yang lebih unggul untuk data Bitcoin. Hamayel & Owda (2021), Andromeda & Winarsih (2025), dan Kaur *et al*. (2025) menemukan bahwa GRU menghasilkan galat yang lebih kecil daripada LSTM. Sebaliknya, Wijaya *et al*. (2025) memperoleh LSTM sedikit lebih baik daripada GRU dengan RMSE 731,72 berbanding 747,57, Seabe *et al*. (2023) mendapatkan Bi-LSTM sebagai model terbaik, sedangkan Hansun *et al*. (2022) melaporkan bahwa akurasi GRU dan Bi-LSTM relatif setara dengan waktu eksekusi GRU yang sedikit lebih baik.

Perbedaan hasil tersebut menunjukkan bahwa keunggulan LSTM atau GRU bergantung pada periode data, variabel masukan, dan pengaturan model yang digunakan. Sebagian besar studi perbandingan juga hanya memakai data harga dan menyimpulkan model terbaik dari selisih nilai metrik semata, tanpa menguji apakah selisih tersebut signifikan secara statistik maupun stabil terhadap inisialisasi bobot acak. Di sisi lain, penelitian berbasis data *on-chain* umumnya hanya menerapkan satu model, dan An *et al*. (2023) secara eksplisit menyarankan agar penelitian lanjutan membandingkan kinerja berbagai model *deep learning*. Oleh karena itu,

penelitian ini membandingkan LSTM dan GRU secara adil, yaitu dengan data, panjang *window*, *grid hyperparameter*, dan *seed* yang identik, menggunakan harga penutupan dan sembilan variabel informasi *blockchain*. Perbedaan akurasi kedua model kemudian diuji dengan uji Diebold-Mariano (Diebold & Mariano, 1995) dan uji kestabilan lintas *seed*. Kombinasi inilah yang menjadi kebaruan (*novelty*) dalam penelitian ini.

Berdasarkan uraian di atas, peneliti tertarik untuk melakukan penelitian yang berjudul "Perbandingan Metode *Long Short-Term Memory* dan *Gated Recurrent Unit* dalam Memprediksi Harga *Bitcoin* Berdasarkan Informasi *Blockchain*". Penelitian ini diharapkan dapat memberikan kontribusi terhadap literatur analisis aset kripto serta menyediakan dasar kuantitatif bagi investor dan praktisi manajemen risiko dalam memilih model prediksi harga Bitcoin yang akurat sekaligus efisien.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana mengimplementasikan dan mengoptimalkan model *Long Short-Term Memory* (LSTM) dan *Gated Recurrent Unit* (GRU) untuk memprediksi harga *Bitcoin* berdasarkan informasi *blockchain*?
2. Bagaimana perbandingan akurasi model LSTM dan GRU dalam memprediksi harga *Bitcoin*, dan apakah perbedaan akurasi keduanya signifikan secara statistik?
3. Bagaimana perbandingan efisiensi (jumlah parameter dan waktu pelatihan) serta kestabilan model LSTM dan GRU dalam memprediksi harga *Bitcoin*?

1.3 Tujuan Penelitian

Sejalan dengan rumusan masalah yang telah ditetapkan, tujuan dari penelitian ini adalah sebagai berikut:

1. Untuk mengimplementasikan dan mengoptimalkan model LSTM dan GRU dalam memprediksi harga *Bitcoin* berdasarkan informasi *blockchain* melalui penentuan jumlah *neuron* dan *epoch* terbaik.
2. Untuk membandingkan akurasi model LSTM dan GRU menggunakan metrik *Root Mean Square Error* (RMSE), *Mean Absolute Error* (MAE), *Mean Absolute Percentage Error* (MAPE), dan akurasi arah, serta menguji signifikansi perbedaannya dengan uji Diebold-Mariano.
3. Untuk membandingkan efisiensi dan kestabilan model LSTM dan GRU berdasarkan jumlah parameter, waktu pelatihan, dan hasil pelatihan ulang dengan lima *seed* yang berbeda.

1.4 Batasan Masalah

Adapun batasan masalah dalam penelitian ini adalah sebagai berikut:

1. Data yang digunakan adalah harga penutupan (*close price*) harian *Bitcoin* dalam USD yang berperan sebagai variabel target sekaligus fitur, serta sembilan variabel informasi *blockchain*, yaitu *Miners Revenue, Difficulty, Hash Rate, Median Confirmation Time, Average Block Size, Total Unique Addresses, Transaction per Block, Confirmed Transaction*, dan *Cost % per Transaction*.
2. Data diambil dari Blockchain.com Charts API dengan frekuensi harian pada periode 1 Juli 2023 s.d. 31 Juli 2026 (1.127 hari).
3. Model yang dibandingkan hanya LSTM dan GRU dengan satu lapisan rekuren dan satu lapisan *dense* keluaran. *Hyperparameter* yang dioptimalkan terbatas pada jumlah *neuron* {10, 20, 30, 40, 50} dan jumlah *epoch* {100, 500, 1000}.
4. Prediksi dilakukan untuk satu hari ke depan (*one-step ahead*) berdasarkan informasi tujuh hari sebelumnya, dan tidak mencakup peramalan jangka panjang di luar periode data uji.
5. Faktor eksternal seperti sentimen pasar, regulasi, dan variabel makroekonomi tidak dimasukkan ke dalam model.

1.5 Landasan Teori

1.5.1 Bitcoin

Bitcoin merupakan mata uang digital terdesentralisasi yang beroperasi pada jaringan pembayaran *peer-to-peer* (P2P) yang bersifat *open source*. P2P adalah model jaringan komputer yang terdiri atas dua atau lebih komputer (*node*) yang dapat saling berbagi data, sehingga pengguna dapat bertransaksi secara langsung tanpa memerlukan jasa pihak ketiga seperti bank (Nakamoto, 2008). Unsur-unsur Bitcoin adalah jaringan *peer-to-peer*, blok, *blockchain*, dan penambang (*miners*). Transaksi yang terjadi di jaringan disimpan dalam berkas yang disebut blok, blok-blok tersebut saling terhubung membentuk rantai blok (*blockchain*), dan penambang memecahkan persoalan kriptografis untuk memvalidasi blok baru sekaligus memperoleh imbalan berupa Bitcoin baru dan biaya transaksi (Muhammad, 2024). Karena tidak ada otoritas pusat yang menetapkan nilainya, harga Bitcoin sepenuhnya ditentukan oleh penawaran dan permintaan di pasar.

1.5.2 Teknologi *Blockchain* dan Informasi *Blockchain*

Fondasi teknologi yang memungkinkan keberadaan Bitcoin adalah *blockchain*. *Blockchain* dapat dipahami sebagai buku besar digital (*digital ledger*) yang terdistribusi, terdesentralisasi, dan tidak dapat diubah (*immutable*). Setiap

transaksi dikelompokkan ke dalam blok, kemudian setiap blok baru dihubungkan secara kriptografis ke blok sebelumnya sehingga membentuk rantai yang kronologis dan aman. Salinan buku besar disimpan oleh banyak *node* di seluruh dunia sehingga tidak ada satu entitas pun yang dapat mengubah catatan transaksi secara sepihak (Muhammad, 2024).

Transparansi *blockchain* publik membuka peluang analisis *on-chain*, yaitu analisis terhadap data yang tercatat langsung di *blockchain*. Berbeda dengan data *off-chain* seperti berita atau sentimen media sosial, data *on-chain* memberikan gambaran langsung mengenai aktivitas ekonomi, kesehatan jaringan, dan perilaku kolektif partisipan (Chrisna *et al*., 2024). An *et al*. (2023) mengelompokkan data *on-chain* ke dalam beberapa kategori, antara lain arus dana di bursa, arus dana penambang, indikator pasar, dan data jaringan. Informasi *blockchain* yang digunakan dalam penelitian ini dapat dikelompokkan sebagai berikut:

- **Metrik penambangan dan keamanan jaringan**, yaitu *Miners Revenue*, *Difficulty*, dan *Hash Rate*. *Hash rate* yang tinggi menunjukkan jaringan yang kuat dan aman serta mencerminkan kepercayaan penambang terhadap profitabilitas jangka panjang (Chrisna *et al*., 2024). Nilai *difficulty* disesuaikan oleh jaringan setiap 2.016 blok (sekitar dua minggu), sehingga pada data harian nilainya berbentuk fungsi tangga (*step function*).
- **Metrik aktivitas dan penggunaan jaringan**, yaitu *Total Unique Addresses*, *Confirmed Transaction*, *Transaction per Block*, dan *Average Block Size*, yang mencerminkan tingkat adopsi dan aktivitas ekonomi pada jaringan Bitcoin.
- **Metrik biaya dan kepadatan jaringan**, yaitu *Median Confirmation Time* dan *Cost % per Transaction*, yang menggambarkan waktu tunggu konfirmasi transaksi serta besarnya biaya jaringan relatif terhadap volume transaksi.

1.5.3 *Time Series*

Time series atau runtun waktu adalah kumpulan data yang diurutkan berdasarkan waktu dan digunakan untuk menganalisis pola hubungan antara suatu variabel dengan waktu. Analisis runtun waktu banyak digunakan untuk peramalan, seperti memprediksi harga aset, nilai tukar, atau penjualan. Data runtun waktu dapat memiliki pola horizontal (fluktuasi acak), pola tren (kecenderungan naik atau turun dalam jangka panjang), pola musiman (fluktuasi berulang secara periodik), dan pola siklis (fluktuasi dalam periode yang lebih panjang). Data harga aset kripto umumnya bersifat nonlinier dan nonstasioner sehingga menyulitkan pembentukan prediksi yang tepat dengan metode linier (Seabe *et al*., 2023). Karena urutan waktu mengandung informasi, data runtun waktu tidak boleh diacak ketika dibagi menjadi data latih dan data uji.

1.5.4 Statistik Deskriptif dan Korelasi

Statistik deskriptif digunakan untuk mengenali karakteristik data sebelum dimodelkan. Ukuran pemusatan yang digunakan adalah rata-rata, sedangkan ukuran penyebaran yang digunakan adalah standar deviasi sampel yang dirumuskan sebagai berikut:

$$\overline{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \tag{1}$$

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}\left(x_i - \overline{x}\right)^2} \tag{2}$$

Keeratan hubungan linier antara setiap variabel informasi *blockchain* dan harga penutupan diukur dengan koefisien korelasi Pearson:

$$r_{xy} = \frac{\sum_{i=1}^{n}\left(x_i - \overline{x}\right)\left(y_i - \overline{y}\right)}{\sqrt{\sum_{i=1}^{n}\left(x_i - \overline{x}\right)^2}\sqrt{\sum_{i=1}^{n}\left(y_i - \overline{y}\right)^2}} \tag{3}$$

Keterangan :

x_i = Nilai observasi ke-i
$\overline{x}$ = Rata-rata sampel
s = Standar deviasi sampel
n = Banyaknya observasi
r_{xy} = Koefisien korelasi Pearson antara variabel x dan y, bernilai -1 sampai $+1$

Nilai koefisien korelasi Pearson yang diperoleh kemudian diinterpretasikan berdasarkan kriteria keeratan hubungan yang disajikan pada Tabel 1.

Tabel 1. Interpretasi Koefisien Korelasi Pearson

r	**Interpretasi**
−1	Korelasi negatif sangat tinggi (sempurna)
−0,99 s.d. −0,81	Korelasi negatif tinggi
−0,80 s.d. −0,61	Korelasi negatif cukup
−0,60 s.d. −0,41	Korelasi negatif agak rendah
−0,40 s.d. −0,21	Korelasi negatif rendah
−0,20 s.d. −0,01	Korelasi negatif sangat rendah
0	Tidak berkorelasi
0,01 s.d. 0,20	Korelasi positif sangat rendah
0,21 s.d. 0,40	Korelasi positif rendah
0,41 s.d. 0,60	Korelasi positif agak rendah

r	Interpretasi
0,61 s.d. 0,80	Korelasi positif cukup
0,81 s.d. 0,99	Korelasi positif tinggi
1	Korelasi positif sangat tinggi (sempurna)

1.5.5 Data *Preprocessing*

Preprocessing data merupakan tahap persiapan data sebelum memasuki proses pelatihan model. Tahap ini meliputi pembersihan data, penanganan nilai hilang, normalisasi, dan pembagian data agar data yang digunakan siap dan mendukung proses pelatihan secara optimal (Wijaya *et al*., 2025).

1.5.5.1 Data *Cleaning*

Data *cleaning* adalah proses untuk menangani data yang tidak lengkap atau mengandung *noise*. *Missing value* terjadi ketika terdapat bagian dataset yang kosong. Pada data runtun waktu harian, penghapusan baris akan memutus keteraturan urutan waktu, sedangkan pengisian dengan rata-rata keseluruhan atau interpolasi linier memakai nilai hari sesudahnya sehingga menimbulkan kebocoran informasi masa depan (*look-ahead bias*). Oleh karena itu, penelitian ini menggunakan metode *forward fill*, yaitu mengisi nilai yang kosong dengan nilai hari sebelumnya:

$$x_{j,t} = x_{j,t-1}, \text{jika } x_{j,t} \text{ kosong} \tag{4}$$

Keterangan :

$x_{j,t}$ = Nilai variabel ke-j pada hari ke-t

$x_{j,t-1}$ = Nilai variabel ke-j pada hari sebelumnya

Metode ini sesuai untuk variabel *blockchain* yang bersifat *persistent*, yaitu berubah lambat dari hari ke hari, dan hanya memakai informasi masa lalu sehingga bebas dari *look-ahead bias*.

1.5.5.2 Pembagian Data Latih, Validasi, dan Uji

Pembagian data adalah proses membagi dataset menjadi data latih (*training*), data validasi (*validation*), dan data uji (*testing*). Data latih digunakan untuk memperbarui bobot jaringan, data validasi digunakan untuk memilih *hyperparameter* terbaik, dan data uji digunakan untuk mengukur kinerja akhir model pada data yang belum pernah dilihat. Pada data runtun waktu, pembagian dilakukan secara kronologis tanpa pengacakan. Dengan proporsi 80% data awal untuk pelatihan (yang 10% terakhirnya menjadi data validasi) dan 20% data akhir untuk pengujian, ukuran setiap bagian dihitung sebagai berikut:

$$n_{\text{lp}} = \lfloor 0{,}8 \times n \rfloor \; n_{\text{uji}} = n - n_{\text{lp}}$$
$$n_{\text{val}} = \lfloor 0{,}1 \times n_{\text{lp}} \rfloor \; n_{\text{latih}} = n_{\text{lp}} - n_{\text{val}} \quad (5)$$

Keterangan :

n	= Jumlah seluruh observasi
n_{lp}	= Jumlah observasi 80% data awal (latih penuh)
$n_{\text{latih}}, n_{\text{val}}, n_{\text{uji}}$	= Jumlah observasi data latih, validasi, dan uji
$\lfloor \cdot \rfloor$	= Pembulatan ke bawah (*floor*)

1.5.5.3 Normalisasi Data

Normalisasi data bertujuan menyamakan skala data sehingga nilai suatu fitur tidak terlalu jauh berbeda dengan fitur lainnya. Dengan normalisasi, risiko bias terhadap satu fitur dapat diminimalkan sehingga model dapat belajar dengan lebih efektif (Fitrinanda & Djunaidy, 2022). Metode yang digunakan adalah *min-max scaling* yang mengubah data ke rentang [0, 1] dengan persamaan berikut:

$$x'_{j,t} = \frac{x_{j,t} - x_{j,\min}}{x_{j,\max} - x_{j,\min}} \quad (6)$$

Keterangan :

$x'_{j,t}$	= Nilai variabel ke-j pada hari ke-t setelah dinormalisasi
$x_{j,t}$	= Nilai aktual variabel ke-j pada hari ke-t
$x_{j,\min}$	= Nilai terendah variabel ke-j pada data latih
$x_{j,\max}$	= Nilai tertinggi variabel ke-j pada data latih

Nilai $x_{j,\min}$ dan $x_{j,\max}$ hanya dihitung dari data latih. Apabila penskala dihitung dari seluruh data, informasi dari data uji, misalnya harga tertinggi yang baru terjadi pada periode uji, akan ikut masuk ke proses pelatihan. Peristiwa ini disebut kebocoran data (*data leakage*) dan membuat hasil evaluasi menjadi terlalu optimistis. Konsekuensinya, nilai ternormalisasi pada data validasi dan uji dapat sedikit keluar dari rentang [0, 1].

1.5.5.4 Pembentukan *Sliding Window*

Lapisan rekuren membutuhkan masukan berbentuk tiga dimensi, yaitu (jumlah sampel, panjang *window*, jumlah fitur). Dengan panjang *window* (*lookback*) tujuh hari, harga penutupan hari $t+1$ diprediksi menggunakan variabel pada tujuh hari sebelumnya:

$$\mathbf{X}^{(t)} = \left[\mathbf{x}_{t-6}; \mathbf{x}_{t-5}; \ldots; \mathbf{x}_t\right] \in \mathrm{R}^{7 \times 10}, y^{(t)} = x'_{1,t+1} \quad (7)$$

Keterangan :

$\mathbf{X}^{(t)}$ = Matriks masukan berisi variabel ternormalisasi selama 7 hari

$\mathbf{x}_t$ = Vektor variabel ternormalisasi pada hari ke-t

$y^{(t)}$ = Target, yaitu harga penutupan ternormalisasi hari ke-$(t+1)$

1.5.5.5 Denormalisasi Data

Setelah prediksi diperoleh, nilai yang masih berada pada skala ternormalisasi dikembalikan ke skala aslinya melalui denormalisasi, yaitu kebalikan dari persamaan (6), dengan nilai minimum dan maksimum yang sama dengan saat normalisasi:

$$\hat{y}_t = \hat{y}'_t \times \left(x_{1,\max} - x_{1,\min}\right) + x_{1,\min} \quad (8)$$

Keterangan :

$\hat{y}'_t$ = Hasil prediksi harga pada hari ke-t sebelum denormalisasi

$\hat{y}_t$ = Hasil prediksi harga pada hari ke-t setelah denormalisasi

$x_{1,\min}, x_{1,\max}$ = Harga penutupan terendah dan tertinggi pada data latih

1.5.6 *Deep Learning*

Deep Learning merupakan salah satu bidang dari *Machine Learning* yang memanfaatkan jaringan saraf tiruan berlapis untuk menyelesaikan permasalahan dengan dataset yang besar. Dengan menambahkan lebih banyak lapisan, model dapat mempelajari representasi data yang bertingkat dan nonlinier tanpa memerlukan rekayasa fitur secara manual. Salah satu keluarga algoritma *deep learning* yang terbukti berhasil digunakan untuk memprediksi data *time series* adalah *Recurrent Neural Network* (RNN) beserta turunannya, yaitu *Long Short Term Memory* (LSTM) dan *Gated Recurrent Unit* (GRU), yang banyak diterapkan untuk memprediksi harga aset kripto (Hansun *et al*., 2022; Seabe *et al*., 2023).

1.5.7 *Neural Network* dan *Recurrent Neural Network*

Neural Network adalah model yang terinspirasi oleh cara kerja neuron di otak manusia. *Neural Network* terdiri atas sejumlah elemen pemrosesan informasi (neuron) yang saling terhubung dan bekerja bersama untuk menyelesaikan masalah klasifikasi maupun prediksi. Jaringan tidak diprogram untuk menghasilkan keluaran tertentu, melainkan belajar dari pola-pola masukan dengan menyesuaikan bobot setiap *node* selama proses pembelajaran (Ridho *et al*., 2022). Setiap neuron menghitung kombinasi linier dari masukannya, kemudian menerapkan fungsi aktivasi:

$$a = \varphi\left(\sum_{j=1}^{m} w_j x_j + b\right) \tag{9}$$

Keterangan :

x_j = Masukan ke-j
w_j = Bobot (*weight*) masukan ke-j
b = Bias
φ = Fungsi aktivasi
a = Keluaran neuron

Recurrent Neural Network (RNN) adalah jaringan saraf yang memiliki koneksi berulang, yaitu keluaran pada *time step* sebelumnya menjadi masukan pada *time step* berikutnya sehingga mampu memodelkan data sekuensial:

$$\mathbf{h}_t = \tanh\left(\mathbf{x}_t \mathbf{W}_h + \mathbf{h}_{t-1} \mathbf{U}_h + \mathbf{b}_h\right) \tag{10}$$

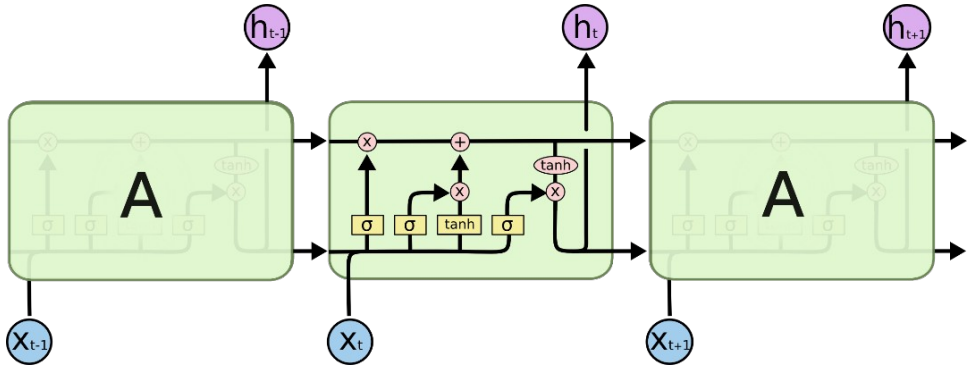
Keterangan :

$\mathbf{h}_t$ = *Hidden state* pada *time step* ke-t
$\mathbf{x}_t$ = Vektor masukan pada *time step* ke-t
$\mathbf{W}_h, \mathbf{U}_h$ = Matriks bobot masukan dan bobot rekuren
$\mathbf{b}_h$ = Vektor bias

Ketika dilatih dengan *backpropagation through time*, gradien pada RNN sederhana cenderung mengecil secara eksponensial (*vanishing gradient*) atau membesar tak terkendali (*exploding gradient*) pada urutan yang panjang, sehingga RNN sulit mempelajari ketergantungan jangka panjang (Fitrinanda & Djunaidy, 2022). LSTM dan GRU dikembangkan untuk mengatasi masalah tersebut melalui mekanisme gerbang (*gate*).

1.5.8 *Long Short Term Memory*

Long Short-Term Memory (LSTM) adalah salah satu bentuk RNN yang pertama kali diperkenalkan oleh Sepp Hochreiter dan Jürgen Schmidhuber pada tahun 1997 (Hochreiter & Schmidhuber, 1997). LSTM menggantikan neuron pada lapisan tersembunyi RNN dengan sel memori yang dilengkapi tiga jenis gerbang, yaitu *forget gate*, *input gate*, dan *output gate*. Mekanisme ini membuat LSTM efektif dalam memproses dan memprediksi data *time series* yang memiliki ketergantungan jangka panjang (Ramadhan, 2024). Struktur LSTM dijelaskan oleh Olah (2015) dan dapat dilihat pada Gambar 1.



Gambar 1. Struktur LSTM

Sumber: Olah, 2015. https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Selain tiga gerbang utama, setiap sel memori dilengkapi tiga lapisan sigmoid dan satu lapisan tanh. Lapisan sigmoid menghasilkan nilai antara 0 dan 1 yang menunjukkan seberapa banyak informasi yang dipertahankan atau dibuang. Nilai 0 berarti "tidak ada informasi yang diperbolehkan lewat", sedangkan nilai 1 berarti "biarkan semua informasi lewat" (Ramadhan, 2024). Fungsi aktivasi sigmoid dinyatakan pada persamaan berikut:

$$\sigma(z)=\frac{1}{1+e^{-z}} \tag{11}$$

Lapisan tanh mengatur nilai agar berada dalam rentang -1 hingga 1 sehingga informasi baru yang masuk ke sel memori tetap stabil dan terkendali selama proses pembelajaran. Fungsi aktivasi tanh didefinisikan sebagai berikut:

$$\tanh(z)=\frac{e^{z}-e^{-z}}{e^{z}+e^{-z}} \tag{12}$$

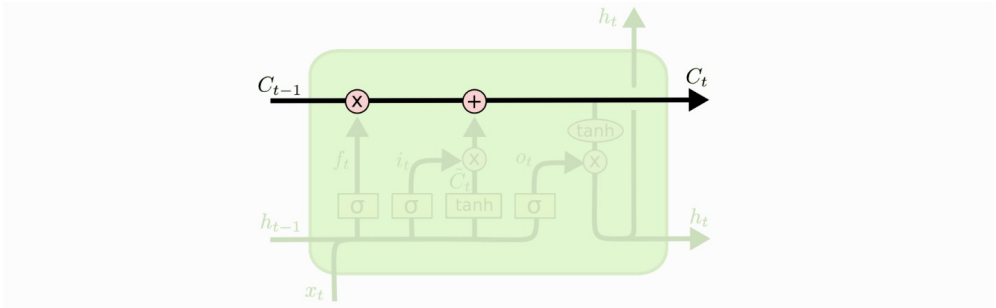
Jika fungsi tanh dihubungkan dengan fungsi sigmoid pada persamaan (11), maka hubungan tersebut dapat dinyatakan melalui persamaan berikut:

$$\tanh(z)=2\sigma(2z)-1 \tag{13}$$

Keterangan :

$\sigma(z)$ = Fungsi aktivasi sigmoid
z = Nilai pra-aktivasi (*input* fungsi aktivasi)
e = Bilangan Euler ($\approx 2{,}71828$)

Kunci utama LSTM adalah *cell state*, yaitu jalur horizontal yang menghubungkan *cell state* lama $\mathbf{c}_{t-1}$ dengan *cell state* baru $\mathbf{c}_t$. Jalur ini memungkinkan informasi penting mengalir dari satu langkah ke langkah berikutnya dengan sedikit modifikasi, sehingga LSTM mampu mengenali pola yang bergantung pada hubungan jangka panjang. *Cell state* dapat dilihat pada Gambar 2.



Gambar 2. *Cell State*

Sumber: Olah, 2015. https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Sebelum pelatihan, bobot diinisialisasi secara acak. Pada implementasi Keras, matriks bobot masukan diinisialisasi dengan metode Glorot *uniform* (Glorot & Bengio, 2010), matriks bobot rekuren dengan matriks ortogonal, dan bias dengan nol, kecuali bias *forget gate* LSTM yang diberi nilai satu (*unit forget bias*) agar memori jangka panjang tidak langsung terlupakan di awal pelatihan (Chollet *et al*., 2015). Batas inisialisasi Glorot *uniform* adalah:

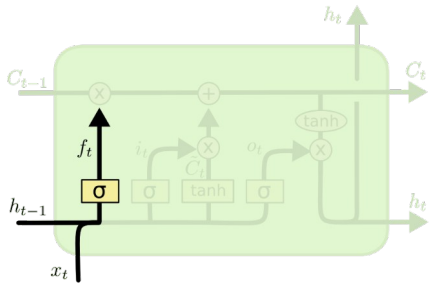
$$\mathbf{W} \sim U\left(-\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}};\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}\right) \tag{14}$$

Keterangan :

$U(a;b)$ = Distribusi seragam pada selang (a,b)

$n_{\text{in}}, n_{\text{out}}$ = Banyaknya baris dan kolom matriks bobot (*fan-in* dan *fan-out*)

Langkah pertama dalam proses LSTM adalah menentukan informasi yang dibuang dari *cell state* melalui lapisan sigmoid yang disebut *forget gate*. *Forget gate* menerima masukan saat ini $\mathbf{x}_t$ dan keluaran *time step* sebelumnya $\mathbf{h}_{t-1}$, lalu menghasilkan nilai antara 0 dan 1 untuk setiap elemen $\mathbf{c}_{t-1}$ (Gambar 3). Nilai mendekati 0 berarti informasi dibuang, sedangkan nilai mendekati 1 berarti informasi dipertahankan (Ramadhan, 2024).



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

Gambar 3. Struktur LSTM *Forget Gate*

Sumber: Olah, 2015. https://colah.github.io/posts/2015-08-Understanding-LSTMs/

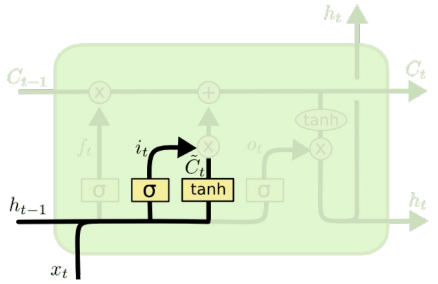
Dalam penelitian ini, persamaan LSTM ditulis dalam notasi vektor baris sesuai implementasi Keras, yaitu $\mathbf{x}_t\mathbf{W}_f+\mathbf{h}_{t-1}\mathbf{U}_f$. Penulisan ini setara dengan notasi $\mathbf{W}_f\left[\mathbf{h}_{t-1},\mathbf{x}_t\right]$ pada Olah (2015) karena matriks bobot gabungan dapat dipecah menjadi bagian untuk $\mathbf{x}_t$ dan bagian untuk $\mathbf{h}_{t-1}$. *Forget gate* dirumuskan sebagai berikut:

$$\mathbf{f}_t=\sigma\left(\mathbf{x}_t\mathbf{W}_f+\mathbf{h}_{t-1}\mathbf{U}_f+\mathbf{b}_f\right) \tag{15}$$

Keterangan :

$\mathbf{f}_t$	= *Forget gate* pada *time step* ke-*t*
$\mathbf{W}_f$	= Matriks bobot masukan *forget gate* (berukuran jumlah fitur × jumlah *neuron*)
$\mathbf{U}_f$	= Matriks bobot rekuren *forget gate* (berukuran jumlah *neuron* × jumlah *neuron*)
$\mathbf{h}_{t-1}$	= *Hidden state* (keluaran) pada *time step* ke-$(t-1)$
$\mathbf{x}_t$	= Vektor masukan pada *time step* ke-*t*
$\mathbf{b}_f$	= Vektor bias *forget gate*

Langkah selanjutnya adalah memutuskan informasi baru yang disimpan dalam *cell state*. Lapisan sigmoid yang disebut *input gate* $\mathbf{i}_t$ menentukan nilai yang perlu diperbarui, sedangkan lapisan tanh membentuk vektor kandidat $\tilde{\mathbf{c}}_t$ yang siap ditambahkan ke *cell state* (Gambar 4).



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Gambar 4. Struktur LSTM *Input Gate*

Sumber: Olah, 2015. https://colah.github.io/posts/2015-08-Understanding-LSTMs/

$$\mathbf{i}_t=\sigma\left(\mathbf{x}_t\mathbf{W}_i+\mathbf{h}_{t-1}\mathbf{U}_i+\mathbf{b}_i\right) \tag{16}$$

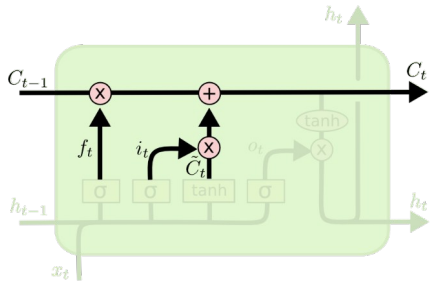
$$\tilde{\mathbf{c}}_t=\tanh\left(\mathbf{x}_t\mathbf{W}_c+\mathbf{h}_{t-1}\mathbf{U}_c+\mathbf{b}_c\right) \tag{17}$$

Keterangan :

$\mathbf{i}_t$	= *Input gate* pada *time step* ke-*t*
$\tilde{\mathbf{c}}_t$	= Kandidat nilai baru untuk *cell state*
$\mathbf{W}_i,\mathbf{U}_i,\mathbf{b}_i$	= Bobot masukan, bobot rekuren, dan bias *input gate*
$\mathbf{W}_c,\mathbf{U}_c,\mathbf{b}_c$	= Bobot masukan, bobot rekuren, dan bias kandidat *cell state*

Pembaruan *cell state* lama dilakukan dengan mengalikan $\mathbf{c}_{t-1}$ dengan *forget gate* sehingga informasi yang tidak relevan terhapus, kemudian menambahkan hasil

perkalian $\mathbf{i}_t$ dengan $\tilde{\mathbf{c}}_t$ (Gambar 5):



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Gambar 5. Struktur LSTM Pembaruan *Cell State*

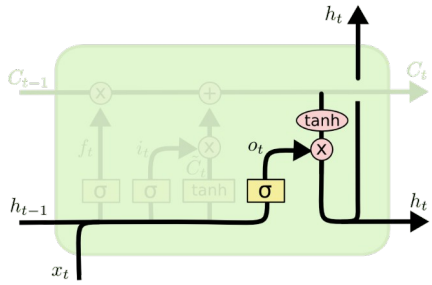
Sumber: Olah, 2015. https://colah.github.io/posts/2015-08-Understanding-LSTMs/

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \tag{18}$$

Keterangan :

$\mathbf{c}_t$	= *Cell state* pada *time step* ke-*t*
$\mathbf{c}_{t-1}$	= *Cell state* pada *time step* ke-$(t-1)$
$\odot$	= Perkalian elemen demi elemen (*Hadamard product*)

Langkah terakhir adalah menentukan keluaran. *Output gate* menentukan bagian *cell state* yang dikeluarkan, kemudian *cell state* diproses dengan fungsi tanh dan dikalikan dengan keluaran *output gate* (Gambar 6):



$$o_t = \sigma\left(W_o\ [h_{t-1}, x_t]\ +\ b_o\right)$$
$$h_t = o_t * \tanh\left(C_t\right)$$

Gambar 6. Struktur LSTM *Output Gate*

Sumber: Olah, 2015. https://colah.github.io/posts/2015-08-Understanding-LSTMs/

$$\mathbf{o}_t = \sigma\left(\mathbf{x}_t \mathbf{W}_o + \mathbf{h}_{t-1} \mathbf{U}_o + \mathbf{b}_o\right) \tag{19}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh\left(\mathbf{c}_t\right) \tag{20}$$

Keterangan :

$\mathbf{o}_t$	= *Output gate* pada *time step* ke-*t*
$\mathbf{W}_o, \mathbf{U}_o, \mathbf{b}_o$	= Bobot masukan, bobot rekuren, dan bias *output gate*
$\mathbf{h}_t$	= *Hidden state* (keluaran sel) pada *time step* ke-*t*

Setelah seluruh *time step* diproses, *hidden state* terakhir $\mathbf{h}_T$ diteruskan ke lapisan *dense* dengan fungsi aktivasi linier untuk menghasilkan nilai prediksi. Karena aktivasinya linier, keluaran dapat berupa nilai kontinu sehingga sesuai untuk tugas

regresi seperti prediksi harga:

$$\hat{y}'_t = \mathbf{h}_T \mathbf{W}_y + b_y \tag{21}$$

Keterangan :

$\mathbf{h}_T$ = *Hidden state* pada *time step* terakhir ($T = 7$)
$\mathbf{W}_y$ = Vektor bobot lapisan *dense*
b_y = Bias lapisan *dense*
$\hat{y}'_t$ = Hasil prediksi pada skala ternormalisasi

Karena LSTM memiliki empat himpunan bobot (*input gate*, *forget gate*, kandidat *cell state*, dan *output gate*), jumlah parameter model yang terdiri atas satu lapisan LSTM dan satu lapisan *dense* dihitung sebagai berikut:

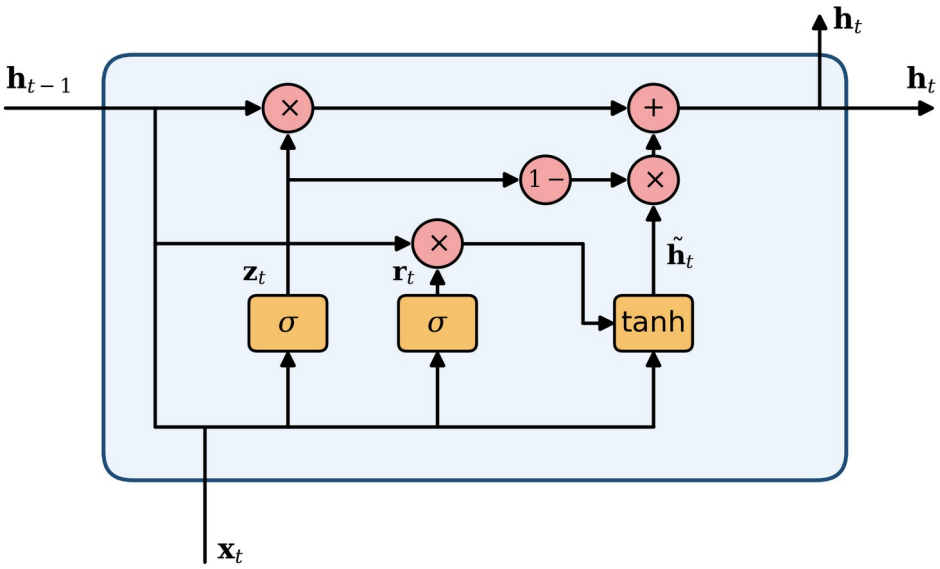
$$P_{\text{LSTM}} = 4\left[n_u\left(n_u + n_f\right) + n_u\right] + \left(n_u + 1\right) \tag{22}$$

Keterangan :

n_u = Jumlah *neuron* (unit) pada lapisan rekuren
n_f = Jumlah fitur masukan

1.5.9 *Gated Recurrent Unit*

Gated Recurrent Unit (GRU) diperkenalkan oleh Cho *et al*. (2014) sebagai penyederhanaan LSTM. GRU menggabungkan *forget gate* dan *input gate* menjadi satu *update gate* serta meniadakan *cell state* terpisah, sehingga memori disimpan langsung pada *hidden state* $\mathbf{h}_t$. Dengan hanya dua gerbang, yaitu *update gate* dan *reset gate*, GRU memiliki parameter yang lebih sedikit dan komputasi yang lebih ringkas, tetapi tetap mampu menangkap ketergantungan temporal pada data berurutan (Chung *et al*., 2014; Wijaya *et al*., 2025). Struktur sel GRU ditunjukkan pada Gambar 7.



Gambar 7. Struktur Sel GRU

Sumber: Olahan penulis berdasarkan Cho et al. (2014)

Update gate $\mathbf{z}_t$ menentukan porsi memori lama yang dipertahankan, sedangkan *reset gate* $\mathbf{r}_t$ menentukan porsi memori lama yang digunakan untuk menyusun kandidat *hidden state* baru. Pada implementasi Keras dengan pengaturan baku reset_after=True, setiap gerbang memiliki dua himpunan bias, yaitu bias masukan dan bias rekuren (Chollet *et al*., 2015). Persamaan GRU yang digunakan dalam penelitian ini adalah sebagai berikut:

$$\mathbf{z}_t = \sigma\left(\mathbf{x}_t \mathbf{W}_z + \mathbf{h}_{t-1} \mathbf{U}_z + \mathbf{b}_z^{(\text{in})} + \mathbf{b}_z^{(\text{rec})}\right) \tag{23}$$

$$\mathbf{r}_t = \sigma\left(\mathbf{x}_t \mathbf{W}_r + \mathbf{h}_{t-1} \mathbf{U}_r + \mathbf{b}_r^{(\text{in})} + \mathbf{b}_r^{(\text{rec})}\right) \tag{24}$$

$$\tilde{\mathbf{h}}_t = \tanh\left(\mathbf{x}_t \mathbf{W}_h + \mathbf{b}_h^{(\text{in})} + \mathbf{r}_t \odot \left(\mathbf{h}_{t-1} \mathbf{U}_h + \mathbf{b}_h^{(\text{rec})}\right)\right) \tag{25}$$

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + \left(1 - \mathbf{z}_t\right) \odot \tilde{\mathbf{h}}_t \tag{26}$$

Keterangan :

$\mathbf{z}_t$	= *Update gate* pada *time step* ke-*t*
$\mathbf{r}_t$	= *Reset gate* pada *time step* ke-*t*
$\tilde{\mathbf{h}}_t$	= Kandidat *hidden state*
$\mathbf{h}_t$	= *Hidden state* pada *time step* ke-*t*
$\mathbf{W}_z, \mathbf{W}_r, \mathbf{W}_h$	= Matriks bobot masukan *update gate*, *reset gate*, dan kandidat
$\mathbf{U}_z, \mathbf{U}_r, \mathbf{U}_h$	= Matriks bobot rekuren *update gate*, *reset gate*, dan kandidat
$\mathbf{b}^{(\text{in})}, \mathbf{b}^{(\text{rec})}$	= Vektor bias masukan dan bias rekuren

Rumus GRU pada makalah asli Cho *et al*. (2014) sedikit berbeda, yaitu gerbang reset diterapkan sebelum perkalian dengan matriks bobot rekuren dan peran $\mathbf{z}_t$ pada pembaruan *hidden state* tertukar:

$$\tilde{\mathbf{h}}_t = \tanh\left(\mathbf{x}_t \mathbf{W}_h + \left(\mathbf{r}_t \odot \mathbf{h}_{t-1}\right)\mathbf{U}_h\right), \mathbf{h}_t = \left(1-\mathbf{z}_t\right)\odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

Kedua bentuk sama-sama sah sebagai GRU, tetapi menghasilkan angka yang berbeda untuk himpunan bobot yang sama. Karena bobot model dilatih menggunakan Keras, perhitungan manual pada Bab III menggunakan persamaan (23) sampai (26). Jumlah parameter model GRU satu lapisan beserta lapisan *dense* adalah:

$$P_{\text{GRU}} = 3\left[n_u\left(n_u + n_f\right) + 2n_u\right] + \left(n_u + 1\right) \tag{27}$$

Angka 3 menyatakan tiga himpunan bobot (*update*, *reset*, dan kandidat), sedangkan suku $2n_u$ berasal dari dua himpunan bias (masukan dan rekuren) akibat pengaturan reset_after=True.

1.5.10 Perbandingan Arsitektur LSTM dan GRU

Perbedaan struktural LSTM dan GRU dirangkum pada Tabel 2. Pada jumlah *neuron* yang sama, GRU selalu memerlukan parameter yang lebih sedikit sehingga secara teoretis lebih hemat memori dan lebih tahan terhadap *overfitting* pada data yang tidak terlalu panjang. Sebaliknya, *cell state* terpisah pada LSTM memberikan kendali memori jangka panjang yang lebih leluasa (Chung *et al*., 2014; Hansun *et al*., 2022).

Tabel 2. Perbandingan Struktur LSTM dan GRU

Aspek	LSTM	GRU
Jumlah gerbang	3 (*forget*, *input*, *output*)	2 (*update*, *reset*)
Memori internal	$\mathbf{h}_t$ dan $\mathbf{c}_t$	Hanya $\mathbf{h}_t$
Himpunan bobot	4 (i, f, c, o)	3 (z, r, h)
Bias per himpunan (Keras)	n_u	$2n_u$
Parameter lapisan rekuren	$4\left[n_u\left(n_u + n_f\right) + n_u\right]$	$3\left[n_u\left(n_u + n_f\right) + 2n_u\right]$

1.5.11 Fungsi *Loss* dan *Adaptive Moment Estimation* (Adam)

Pelatihan jaringan bertujuan meminimumkan fungsi *loss*. Untuk permasalahan regresi, fungsi *loss* yang digunakan adalah *Mean Squared Error* (MSE) pada skala ternormalisasi:

$$L = \frac{1}{N}\sum_{k=1}^{N}\left(y'_k - \hat{y}'_k\right)^2 \tag{28}$$

Keterangan :

L	= Nilai *loss*
$y'_k, \hat{y}'_k$	= Nilai aktual dan prediksi ternormalisasi sampel ke-k
N	= Jumlah sampel

Optimasi berbasis gradien stokastik memainkan peran penting dalam pelatihan jaringan saraf. Salah satu algoritma optimasi yang banyak digunakan adalah *Adaptive Moment Estimation* (Adam) yang menggabungkan kelebihan *Adaptive Gradient* (AdaGrad) dalam menangani gradien jarang dan *Root Mean Square Propagation* (RMSProp) dalam mengatasi perubahan gradien (Kingma & Ba, 2015). Adam menghitung rata-rata bergerak eksponensial dari gradien (momen pertama) dan gradien kuadrat (momen kedua), mengoreksi biasnya, lalu memperbarui parameter:

$$\mathbf{m}_k = \beta_1 \mathbf{m}_{k-1} + (1-\beta_1)\mathbf{g}_k, \mathbf{v}_k = \beta_2 \mathbf{v}_{k-1} + (1-\beta_2)\mathbf{g}_k^2 \tag{29}$$

$$\widehat{\mathbf{m}}_k = \frac{\mathbf{m}_k}{1-\beta_1^k}, \hat{\mathbf{v}}_k = \frac{\mathbf{v}_k}{1-\beta_2^k} \tag{30}$$

$$\boldsymbol{\theta}_k = \boldsymbol{\theta}_{k-1} - \eta \frac{\widehat{\mathbf{m}}_k}{\sqrt{\hat{\mathbf{v}}_k} + \epsilon} \tag{31}$$

Keterangan :

$\mathbf{g}_k$	= Gradien *loss* terhadap parameter pada iterasi ke-k
$\mathbf{m}_k, \mathbf{v}_k$	= Estimasi momen pertama dan momen kedua gradien
$\widehat{\mathbf{m}}_k, \hat{\mathbf{v}}_k$	= Estimasi momen yang telah dikoreksi biasnya
$\boldsymbol{\theta}$	= Parameter model (bobot dan bias)
η	= *Learning rate* (dalam penelitian ini $0{,}001$)
$\beta_1, \beta_2, \epsilon$	= Konstanta peluruhan dan penstabil, bawaan Keras $0{,}9$; $0{,}999$; 10^{-7}

Keunggulan Adam adalah pembaruan parameter yang tidak dipengaruhi oleh penskalaan gradien dan tetap efektif pada data nonstasioner (Ramadhan, 2024). Pada penelitian ini, Adam digunakan dengan pengaturan yang identik untuk LSTM dan GRU sehingga perbedaan hasil hanya berasal dari perbedaan arsitektur.

1.5.12 Evaluasi Model

Evaluasi model adalah langkah untuk mengukur tingkat akurasi model dalam memprediksi data uji. RMSE dan MAE mengukur kesalahan dalam satuan asli data (USD), MAPE mengukur kesalahan dalam bentuk persentase, sedangkan akurasi arah mengukur kemampuan model menebak arah pergerakan harga (Fitrinanda & Djunaidy, 2022; Wijaya *et al*., 2025).

1.5.12.1 *Root Mean Square Error* (RMSE)

RMSE dihitung dengan mengurangkan nilai aktual dengan nilai prediksi, mengkuadratkan selisihnya, merata-ratakan, lalu mengambil akar kuadratnya. Karena selisih dikuadratkan, RMSE memberikan bobot lebih besar pada kesalahan yang besar. Semakin kecil nilai RMSE, semakin baik akurasi model (Fitrinanda & Djunaidy, 2022):

$$RMSE = \sqrt{\frac{1}{n}\sum_{t=1}^{n}\left(y_t - \hat{y}_t\right)^2} \tag{32}$$

1.5.12.2 *Mean Absolute Error* (MAE)

MAE mengukur rata-rata besarnya kesalahan absolut tanpa memperhatikan arah kesalahan, sehingga seluruh kesalahan diperlakukan setara:

$$MAE = \frac{1}{n}\sum_{t=1}^{n}\left|y_t - \hat{y}_t\right| \tag{33}$$

1.5.12.3 *Mean Absolute Percentage Error* (MAPE)

MAPE mengukur persentase kesalahan prediksi relatif terhadap nilai aktual sehingga bebas skala:

$$MAPE = \frac{100\,\%}{n}\sum_{t=1}^{n}\left|\frac{y_t - \hat{y}_t}{y_t}\right| \tag{34}$$

Keterangan :

y_t = Harga aktual pada hari ke-t (USD)
$\hat{y}_t$ = Harga prediksi pada hari ke-t (USD)
n = Jumlah hari pada data uji

Semakin rendah nilai MAPE, semakin akurat prediksi. Kriteria kemampuan peramalan berdasarkan nilai MAPE dapat dilihat pada Tabel 3.

Tabel 3. Kriteria Nilai MAPE

Skala MAPE	Interpretasi
$x < 10\,\%$	Sangat Akurat
$10\,\% \le x < 20\,\%$	Akurat
$20\,\% \le x < 50\,\%$	Cukup
$x \ge 50\,\%$	Tidak Akurat

Sumber: Lewis (1982)

1.5.12.4 Akurasi Arah

Akurasi arah (*directional accuracy*) adalah persentase hari ketika arah perubahan harga (naik atau turun) tertebak dengan benar. Arah aktual dan arah prediksi diukur dari titik acuan yang sama, yaitu harga aktual hari sebelumnya:

$$DA = \frac{100\,\%}{n} \sum_{t=1}^{n} 1\left[\operatorname{sign}\left(y_t - y_{t-1}\right) = \operatorname{sign}\left(\hat{y}_t - y_{t-1}\right)\right] \tag{35}$$

Keterangan :

$1[\cdot]$ = Fungsi indikator, bernilai 1 jika pernyataan benar dan 0 jika salah

$\operatorname{sign}(\cdot)$ = Fungsi tanda (+1 naik, -1 turun)

y_{t-1} = Harga aktual pada hari sebelumnya

Nilai akurasi arah sekitar 50% menunjukkan bahwa model tidak lebih baik daripada menebak arah secara acak, meskipun nilai galatnya dapat saja kecil.

1.5.13 Uji Diebold-Mariano

Selisih nilai metrik antara dua model belum tentu bermakna secara statistik. Uji Diebold-Mariano digunakan untuk menguji apakah dua model peramalan memiliki akurasi yang sama (Diebold & Mariano, 1995). Hipotesis yang diuji adalah $H_0 : E\left[d_t\right] = 0$ (akurasi LSTM dan GRU sama) melawan $H_1 : E\left[d_t\right] \neq 0$ (akurasi berbeda), dengan d_t adalah selisih *loss* kuadrat kedua model:

$$d_t = e_{L,t}^2 - e_{G,t}^2 \tag{36}$$

$$\bar{d} = \frac{1}{T} \sum_{t=1}^{T} d_t \tag{37}$$

$$\gamma_k = \frac{1}{T} \sum_{t=k+1}^{T} \left(d_t - \bar{d}\right)\left(d_{t-k} - \bar{d}\right) \tag{38}$$

$$\hat{V}\left(\bar{d}\right) = \frac{1}{T}\left[\gamma_0 + 2\sum_{k=1}^{h-1} \gamma_k\right] \tag{39}$$

$$DM = \frac{\bar{d}}{\sqrt{\hat{V}\left(\bar{d}\right)}} \tag{40}$$

Harvey *et al*. (1997) mengusulkan koreksi untuk sampel berukuran terbatas yang kemudian dibandingkan dengan distribusi t dengan derajat bebas $T-1$:

$$\mathrm{DM}^{*} = DM \times \sqrt{\frac{T + 1 - 2h + h(h-1)/T}{T}} \tag{41}$$

$$p = 2\left[1 - F_{T-1}\left(\left|\mathrm{DM}^{*}\right|\right)\right] \tag{42}$$

Keterangan :

$e_{L,t}, e_{G,t}$	= Galat prediksi LSTM dan GRU pada hari ke-t
T	= Jumlah pengamatan pada data uji
h	= Horizon peramalan (dalam penelitian ini $h=1$)
γ_k	= Autokovarians d_t pada lag ke-k
F_{T-1}	= Fungsi distribusi kumulatif distribusi t dengan derajat bebas $T-1$

Karena $h=1$, suku penjumlahan autokovarians pada persamaan (39) kosong sehingga $\hat{V}(\bar{d})=\gamma_0/T$. H_0 ditolak apabila $p<0{,}05$. Nilai $\bar{d}>0$ berarti rata-rata *loss* LSTM lebih besar daripada GRU, sedangkan $\bar{d}<0$ berarti sebaliknya.

1.5.14 Penelitian Terdahulu

Beberapa penelitian terdahulu dalam lima tahun terakhir yang relevan dengan penelitian ini dirangkum pada Tabel 4.

Tabel 4. Penelitian Terdahulu

No.	Peneliti (Tahun)	Data dan Metode	Hasil Utama
1.	Hamayel & Owda (2021)	BTC, ETH, LTC; LSTM, GRU, Bi-LSTM	GRU menghasilkan MAPE terkecil pada ketiga aset kripto
2.	Rizkilloh & Widiyanesti (2022)	Harga aset kripto; LSTM	LSTM mampu memprediksi harga aset kripto dengan galat yang rendah
3.	Hansun *et al*. (2022)	Lima aset kripto (multivariat); LSTM, Bi-LSTM, GRU	Akurasi Bi-LSTM dan GRU relatif setara; GRU sedikit lebih cepat
4.	Kim *et al*. (2022)	Data *on-chain* Bitcoin; SAM-LSTM	LSTM ganda berbasis *attention* untuk kelompok variabel *on-chain*
5.	Seabe *et al*. (2023)	BTC, ETH, LTC; LSTM, GRU, Bi-LSTM	Bi-LSTM terbaik dengan MAPE BTC 0,036
6.	An *et al*. (2023)	10 variabel *on-chain* (CryptoQuant); LSTM	Data uji: RMSE 2.162,38; MAPE 3,993%; R^2 0,976
7.	Nirraca & Hartati (2024)	Harga harian BTC; LSTM	Prediksi 7 hari (RMSE 1.304,3) lebih akurat daripada 30 hari
8.	Omole & Enke (2024)	87 metrik *on-chain* (Glassnode); CNN-LSTM, LSTNet, TCN, ARIMA	Seleksi fitur *on-chain* meningkatkan prediksi arah harga Bitcoin
9.	Andromeda & Winarsih (2025)	BTC dan ETH; LSTM, GRU	GRU lebih akurat; MAPE BTC 0,38% (GRU) dan 0,41% (LSTM)
10.	Kaur *et al*. (2025)	BTC, LTC, ETH; LSTM, GRU	GRU mengungguli LSTM; MAPE BTC 0,03540
11.	Wijaya *et al*. (2025)	BTC harian 2010–2024; LSTM, GRU, Prophet, VaR, ES	LSTM terbaik (RMSE 731,72); GRU mendekati (RMSE 747,57)